KrnlSentry: eBPF Syscall Monitoring with ATT&CK-Mapped Detection and Cross-Process Attack-Storyline Correlation for Linux

Praneeth
Department of Computer Science and Engineering
«Institution Name», «City, Country»
praneeth132006@gmail.com

***Abstract*—Host-based intrusion detection on Linux must answer two separable questions: how to observe syscalls completely and cheaply, and how to decide which patterns across those observations constitute an attack. This paper presents KrnlSentry, an agent that attaches eBPF programs to fifteen syscall tracepoints and streams observations to user space over a globally ordered ring buffer, where four behavioural rules — privilege escalation, credential access, process injection and reverse shells — evaluate them and tag each finding with its MITRE ATT&CK technique. Two properties distinguish the design. First, the kernel-resident collector carries no policy whatsoever, so every detection rule is a pure function over a plain event structure and the entire detection surface is unit-testable with no kernel, no privilege and no specialised runner; measured statement coverage of the detection package is 94.8% across 88 tests. Second, an attack-storyline correlator reconstructs process lineage from the complete event stream, bounds each story at its session owner, rolls each alert up to its ATT&CK tactic, and raises a single critical incident carrying an ordered timeline when one process tree spans three or more distinct attacker goals — converting four apparently unrelated queue items into one intrusion while leaving a loud single-tactic workload correctly unescalated. A lossless capture format permits any recorded trace to be replayed through the identical filter, engine and sinks on an unprivileged non-Linux host, which makes rule development, regression testing, retro-hunting and false-positive reproduction possible without access to the monitored machine.**

Keywords—eBPF, host intrusion detection, syscall monitoring, MITRE ATT&CK, alert correlation, Linux kernel security, CO-RE

I. Introduction

An adversary who has obtained code execution on a Linux host performs a recognisable progression: read credentials, escalate privilege, manipulate other processes, and establish an outbound channel. Each stage is ultimately expressed in system calls, and each individual call is, considered alone, indistinguishable from ordinary administration. A process opening a credentials file may be an attacker or a developer; a call to setuid may be an exploit or sudo; an attach via ptrace may be an injector or a debugger on a crash.

The consequence is that a detector which examines one syscall at a time is forced into a choice between silence and noise. The behaviour that actually distinguishes an intrusion is not present in any single observation: it lies in the ordering of calls within one process, and in the breadth of distinct objectives pursued across a group of related processes.

This paper describes KrnlSentry, a host agent built around that observation, and reports on its design, implementation and verification. The contributions are fourfold.

First, a strict architectural separation in which the kernel-resident collector performs no policy evaluation at all. It extracts identity, arguments and a monotonic timestamp, and nothing else. Every judgement about suspicion occurs in user space over a plain event structure whose only dependency is the standard library. The practical result is that the entire detection surface can be exercised with structure literals on any operating system, with no kernel, no privilege and no specialised continuous-integration runner.

Second, a sequence detector for reverse shells that is confirmed rather than inferred. The socket, connect, dup2 and execve calls that constitute a reverse shell are individually unremarkable; the detector fires on their ordering within a single process inside a bounded window, and additionally verifies, using a syscall-exit probe, that the descriptor being duplicated onto the standard streams is the descriptor the socket call returned.

Third, an attack-storyline correlator that reconstructs process lineage from the complete event stream, bounds each story at its session owner, and escalates on the breadth of ATT&CK tactics observed within one process tree rather than on alert volume.

Fourth, a lossless capture-and-replay facility which permits any recorded trace to be evaluated through the identical scope filter, detection engine and output sinks on an unprivileged, non-Linux host.

II. Background and Related Work

A. Limitations of conventional observation mechanisms

Three mechanisms are conventionally available for observing process behaviour on Linux, and each is unsuitable for continuous host monitoring. The ptrace facility halts the traced process at each syscall boundary, imposing substantial overhead, and is trivially detectable by the tracee through the TracerPid field of its own status file. The kernel audit subsystem produces a high-volume event stream whose rule language cannot express a condition spanning several syscalls within one process. Library interposition through LD_PRELOAD is bypassed entirely by a statically linked binary or by a direct syscall invocation.

Extended BPF resolves the structural difficulty by allowing a verified program to execute inside the kernel at the syscall boundary [5], [12]. Filtering occurs before any transfer to user space, there is no context switch per event, and the observed process has no mechanism by which to detect or disable the observer. The lineage of this approach extends to the original BSD packet filter [9].

B. CO-RE versus runtime compilation

Two toolchains dominate eBPF development. The BPF Compiler Collection [7] compiles its kernel programs at run time, which requires a compiler, the LLVM toolchain and kernel headers matching the running kernel to be present on every monitored host. Compile Once – Run Everywhere [3] instead compiles the object once, embeds BPF Type Format relocation data [11], and allows the loader to patch structure field offsets against the running kernel's own type information at load time. KrnlSentry adopts the latter: the deliverable is a single statically linked binary that executes on any kernel from version 5.8 onward with BTF enabled, and requires nothing to be installed on the monitored host.

C. ATT&CK as the classification vocabulary

Findings are labelled with MITRE ATT&CK technique identifiers [1], using sub-techniques wherever one applies, because an identifier such as T1003.008 names the observed behaviour whereas its parent names only a category. Each alert carries both the identifier and its human-readable name so that it may be triaged without an enrichment step. The correlator described in Section V operates one level above this, on tactics [2], because the attacker's objective rather than the method is what distinguishes a multi-stage intrusion from loud but singular activity.

III. System Architecture

Fig. 1 shows the complete pipeline. The design is organised around a single invariant: the detection package depends on nothing but the event package and the standard library. Every other property claimed in this paper follows from it.

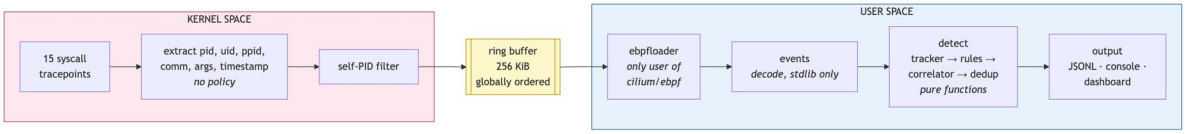


Fig. 1. KrnlSentry architecture. The kernel-resident collector carries no policy; every judgement occurs in user space over a plain event structure whose only dependency is the standard library.

A. Kernel-space collection

Fifteen syscall tracepoints are instrumented: execve and execveat; openat; setuid, setgid, setresuid, setresgid and capset; ptrace; socket and connect at both entry and exit; and dup2 and dup3. Tracepoints are preferred over kprobes because they constitute a stable kernel interface, whereas kprobes placed on syscall symbols break across kernel versions and differ between architectures.

The programs extract the thread and process identifiers, user and group identifiers, parent process identifier, command name, syscall arguments and a monotonic timestamp. They perform no path matching, maintain no allowlist and apply no filtering beyond the suppression of the agent's own process identifier, which breaks the feedback loop in which writing an alert is itself a syscall that produces an event. A per-CPU statistics map counts submitted and dropped events so the agent can report when its own coverage was incomplete rather than under-reporting in silence.

B. Transport

Events are carried on a 256 KiB BPF ring buffer rather than a per-CPU perf event array. The distinction is consequential rather than stylistic: the ring buffer delivers events in a single global order across processors [4], whereas a per-CPU buffer does not. Because the reverse-shell rule is a sequence detector, out-of-order delivery would render it silently incorrect rather than merely untidy, which is the least acceptable failure mode available to a detection system. The reserve-and-commit interface additionally permits the kernel program to populate the event in place, which is necessary because the 544-byte event does not fit within the 512-byte stack the verifier permits.

C. User-space pipeline

A loader drains the ring buffer and is the only component aware of the eBPF library. A decoding package converts raw records into canonical events, naming flags symbolically rather than numerically. A buffered channel of 4096 events decouples draining from evaluation; when it is full, events are discarded at that point and counted, rather than stalling the drain and pushing the loss into the kernel where it would be invisible.

D. The testability invariant

Because detection logic imports only the event package and the standard library, a rule is an ordinary function from an event and that process's recent history to an optional alert. The single impure rule — the check for execution of a set-user-ID binary, which requires a filesystem metadata lookup — receives that lookup as an injected function, so that testing it does not require the creation of a privileged file. Section IX reports the resulting verification coverage.

IV. Detection Methodology

Table I lists the rule set with its ATT&CK mapping and severity assignment. Severity encodes confidence combined with impact rather than the intrinsic danger of the syscall, because its purpose is to order a triage queue.

Because the instrumented tracepoints are placed at syscall entry, a denied attempt produces the same alert as a successful

one. A setuid call that returns EPERM is a privilege-escalation attempt, and an agent that waited for success would detect only those attacks that worked. This property is also what permits the scenario scripts described in Section IX to be safe by construction.

TABLE I.
DETECTION RULES AND ATT&CK MAPPING

Rule	Trigger	ATT&CK	Severity
privilege-escalation	setuid(0), setresuid(0,…), setgid(0), setresgid(0,…) or capset() issued by a non-root process	T1548	HIGH
	execve() of a set-user-ID or set-group-ID binary by a non-root user	T1548.001	HIGH
sensitive-file-access	openat() of /etc/shadow or /etc/gshadow and their backups	T1003.008	HIGH
	openat() of /proc/<pid>/mem	T1003.007	HIGH
	openat() of /proc/<pid>/maps	T1003.007	MEDIUM
	openat() of /etc/sudoers or /etc/sudoers.d/*	T1552	MEDIUM
	openat() of SSH or GnuPG private key material	T1552.004	MEDIUM
	openat() of cloud, container or cluster credential files	T1552.001	MEDIUM
process-injection	ptrace() with ATTACH, SEIZE, POKETEXT, POKEDATA, POKEUSER, SETREGS, SETREGSET or SETFPREGS from a non-allowlisted process	T1055.008	HIGH
reverse-shell	socket → connect → dup2/dup3 onto ≥ 2 standard descriptors → execve, within one process in a 2 s window	T1059	CRITICAL
attack-chain	≥ 3 distinct ATT&CK tactics observed across one session-bounded process tree	all of the above	CRITICAL

A. Sequence detection for reverse shells

Fig. 2 presents the reverse-shell detector as a state machine. None of the four constituent calls is suspicious: every network client issues the first two and every shell redirection issues the third. The detection rests entirely on their order within one process inside a two-second window.

A chain that reaches the final state additionally reports whether the descriptor duplicated onto the standard streams is the descriptor returned by the socket call. Obtaining that return value requires a probe at syscall exit, since the argument registers have been overwritten by the time a syscall returns and only the return value remains meaningful. The confirmation eliminates the coincidental case of a program that opens a socket and separately redirects a file.

B. Bounded process state

A tracker retains recent syscalls per process so that sequence rules may inspect them. It is bounded along three axes, because an unbounded cache within a security agent is itself a denial-of-service primitive: thirty-two entries of history per process, a thirty-second entry lifetime against a longest rule window of two seconds, and a ceiling of 8192 tracked processes swept at five-second intervals.

C. Evaluation order

Fig. 3 shows the order in which one event is processed. Two orderings within it are load-bearing. The tracker is updated before rules execute, so that a rule inspecting history observes the current event as its most recent entry; the reverse-shell rule fires on the execve that completes a chain and requires that execve to be present. Correlation occurs before deduplication, because a folded duplicate remains evidence that a tactic occurred and the storyline must count it even though the operator is not shown it a second time.

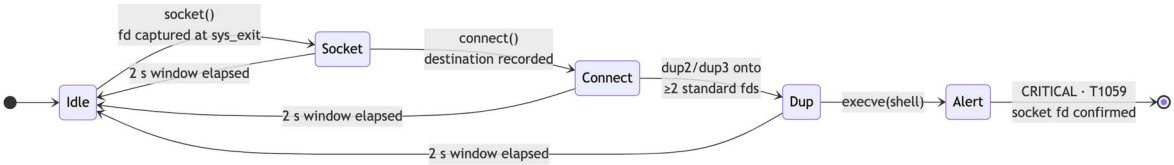


Fig. 2. The reverse-shell detector as a state machine. Each constituent syscall is unremarkable; the detection rests on their order within one process, and the socket descriptor is confirmed against the descriptor duplicated onto the standard streams.

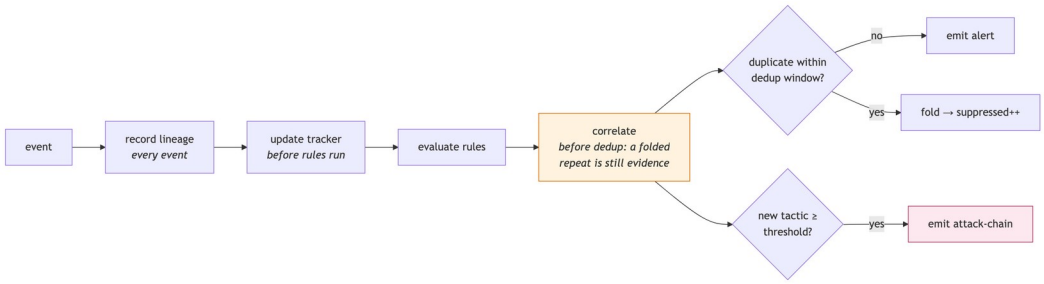


Fig. 3. Evaluation order for a single event. The tracker is updated before rules execute, and correlation precedes deduplication because a folded duplicate remains evidence of a tactic.

V. Attack-Storyline Correlation

Every rule described in Section IV judges a single process at a single moment. Intrusions do not present that way. They present as a login shell whose children, in succession, read credentials, attach to the shell, acquire root and dial outward — four alerts, four process identifiers, four rules, arriving in a queue as four unrelated items.

A. Lineage reconstruction

Lineage is derived from the complete event stream rather than from alerting events alone. Fig. 4 illustrates why: the shell situated between two alerting children is typically the one process that never alerts, and would therefore be absent from a graph built only from alerts. Ancestors that have issued no instrumented syscall since the agent started are recovered from the process filesystem.

B. Session boundaries

From an alerting process the correlator ascends the tree and halts immediately below the process that owns the session — a secure shell daemon, the init system, a container shim, the scheduler, a terminal multiplexer, or process 1. Without this boundary every process on the host shares a common ancestor and the host collapses into a single story; with it, each login session, container and daemon constitutes its own scope.

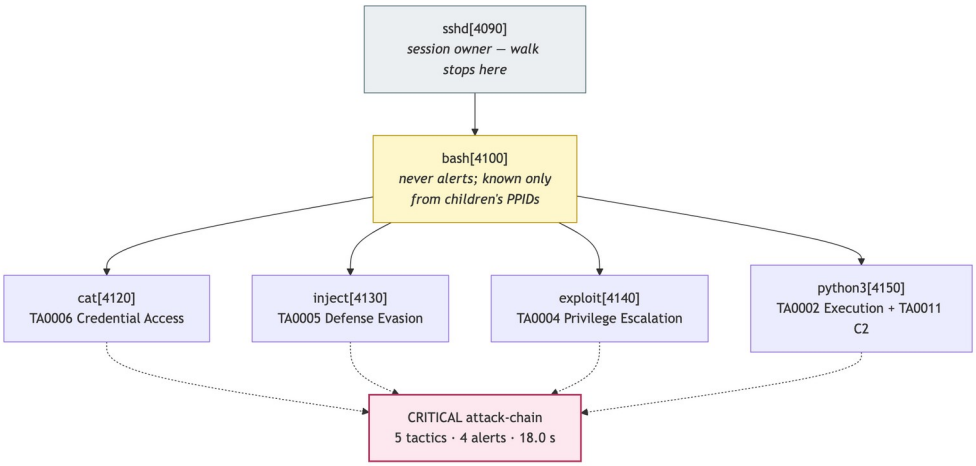


Fig. 4. Lineage reconstruction and escalation. The shell between the alerting children never alerts and is known only from its children's parent identifiers; the walk halts below the session owner.

C. Tactic roll-up and escalation

Each alert's technique is mapped to its ATT&CK tactic, with a completed reverse shell credited to both Execution and Command and Control since it genuinely achieves two objectives. A storyline raises a critical attack-chain alert when it spans a configurable number of distinct tactics, three by default, and raises it again each time a further tactic appears. Escalation therefore responds to breadth, not volume: a scheduled job reading the shadow password file fifty times remains one tactic however loud it is, whereas three distinct objectives within one process tree over a few minutes is the shape of an intrusion and rarely the shape of administration.

D. Output

An attack-chain alert carries an ordered story: one step per contributing alert, each with its millisecond offset from the first, process identifier, command, rule, technique, tactic and a one-line summary. Every alert, of every rule, additionally carries a storyline identifier, so that an analysis platform may pivot from any single alert to all related activity with one equality filter.

VI. Noise Control

A detector that cannot be quietened is a detector that will be disabled. Three independent mechanisms operate at different layers. Scope reduction restricts monitoring to a process tree, a subset of rules, or excludes loopback destinations and self-inspection of the process filesystem. Deduplication folds

identical findings — identified by rule, process, technique and the distinguishing argument — into a single alert per window, which resurfaces carrying a count of what it absorbed; the window is fixed from first emission rather than sliding, so that a steady loop reappears once per window instead of being suppressed indefinitely. Finally, a severity threshold governs what competes for an operator's attention without altering what is recorded: the structured log always receives everything.

VII. Offline Replay and Retro-Hunting

The agent can record every observed syscall to a capture file whose records carry a typed layer alongside the human-readable one. The typed layer is what makes replay lossless: a rule's input is never reconstructed by parsing a presentation string. A capture may then be evaluated through precisely the same scope filter, detection engine and output sinks as live tracing, on a host with no kernel instrumentation and no privilege — the kernel loader is replaced by a stub on non-Linux platforms, so the agent compiles and runs on a development workstation.

Three capabilities follow that live tracing cannot provide: rules may be developed and exercised against real traces without any eBPF environment; a rule written today may be evaluated against a trace recorded last week; and a reported false positive may be reproduced from the reporter's capture rather than from a description of it. A replay may optionally be paced against the capture's own monotonic clock, which is required if a developing storyline is to be observed rather than delivered in a single frame.

VIII. Operator Interface

The default output is one line per alert, which composes with redirection, filtering and remote shells. An optional full-screen display, shown in Fig. 5, addresses the different question an operator asks during an incident: not what the most recent alert was, but how many, how severe, which rules, which processes, whether a chain is forming, and whether events are being lost. On a busy host all of that scrolls out of view within seconds.

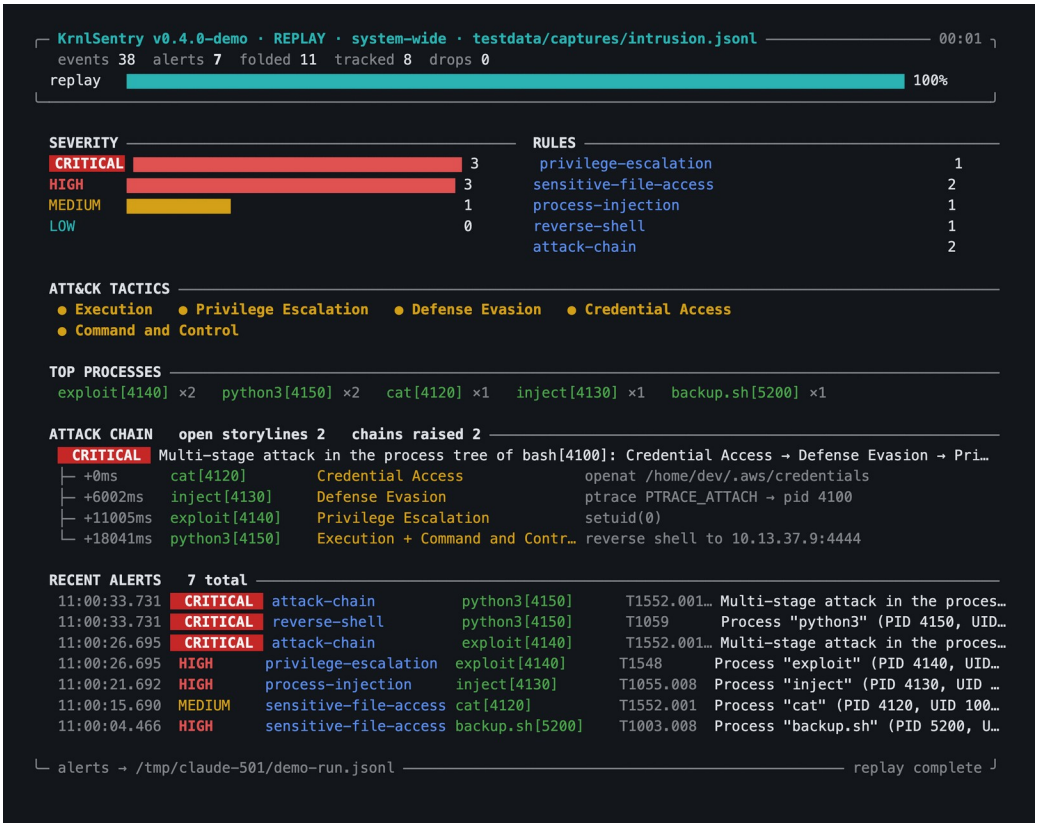


Fig. 5. The operator display at the conclusion of a golden-capture replay. Losses are summed into a single always-visible figure; rules and severities with zero occurrences are retained deliberately.

Four decisions within the display warrant mention. Rules and severities with zero occurrences are retained, because an absent row cannot be distinguished from a rule that is not running. Kernel and queue losses are summed into a single always-visible figure that is highlighted when non-zero, because an agent that silently discarded events is externally indistinguishable from one that observed nothing suspicious. The severity threshold filters the alert feed but never the totals. And the final frame is left on screen at shutdown, so that the run's figures remain legible above the summary.

A repainting display introduces two correctness obligations that are enforced by test. Padding must count printable columns rather than bytes, since a terminal control sequence occupies no display width; and the rendered frame must not exceed the terminal's height, because a frame one line too tall scrolls the window, displacing the origin from which the next repaint

draws. The renderer therefore measures its own output and sheds elastic content — feed rows, then timeline steps, then context panels — until the frame fits, and never sheds the totals.

IX. Verification Methodology

Verification is organised in four layers, shown in Fig. 6, which differ in what they require of the environment and in what they are able to establish.

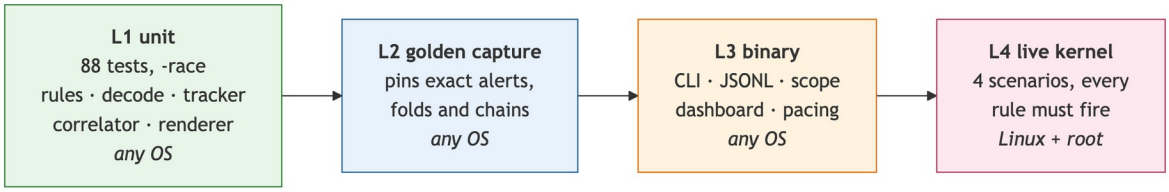


Fig. 6. Four verification layers, differing in what they require of the environment and in what they establish.

The first layer comprises 88 unit tests executed under the race detector, covering rule logic, argument decoding, the process tracker, the correlator, deduplication, the output sinks and the display renderer. None requires a kernel or privilege. The second layer replays a golden capture — a recorded session containing a four-stage intrusion interleaved with benign activity and a deliberately noisy scheduled job — and asserts the exact set of alerts, folds and chains it must produce. The capture is regenerated by the production decoder, so the fixture cannot diverge from the code path it is intended to represent.

The third layer exercises the compiled binary rather than its packages: command-line handling, the structured log and its file permissions, scope filtering, replay pacing equivalence and the display renderer, including its refusal to paint into a redirect. The fourth layer runs four scenario scripts against a live kernel and requires every rule to fire and the correlator to assemble them; it exits unsuccessfully if any rule is silent, so it serves as a smoke test rather than only a demonstration. A further test parses the shared kernel header and fails if the C enumeration and the corresponding constants diverge, so that the two views of the wire format cannot silently disagree.

Table II.
Implementation and Verification Metrics

Metric	Value
Go source (excl. tests, generated)	6 914 lines
Go test source	3 251 lines
eBPF C and headers	1 243 lines
Shell verification harness	1 027 lines
Syscall tracepoints instrumented	15
Detection rules / ATT&CK techniques	5 / 13
Automated tests (race detector)	88, ≈ 3 s
Coverage: detect / events / output	94.8 / 86.4 / 75.9%
Coverage: whole module	63.0%
Static + shell analysis findings	0 + 0
Third-party Go dependencies	2

Table III.
Golden-Capture Replay Results

Rule	Alerts emitted
privilege-escalation	1
sensitive-file-access	2
process-injection	1
reverse-shell	1
attack-chain	2
Total emitted (18 raw hits)	7 + 11 folded

Table IV.
Live-Kernel Scenario Results

Rule	Alerts observed
privilege-escalation	6
sensitive-file-access	17
process-injection	1
reverse-shell	1
attack-chain	2
Total written to the alert log	25

X. Results

Table II reports implementation and verification metrics. Statement coverage of the detection package — the component in which every security decision is made — is 94.8%; the lower figure for the module as a whole reflects the kernel loader, which cannot execute off Linux, and command-line wiring.

Table III reports the alerts produced by replaying the golden capture. The capture contains 38 events spanning 33.7 seconds. Four findings across four processes, spanning five tactics, produce one storyline and a critical attack-chain incident that is raised again when the fifth tactic appears. The scheduled job reading the shadow password file repeatedly contributes a single tactic and no chain, confirming that volume alone cannot escalate. Eighteen raw rule hits are reported as seven alerts and eleven folds. Restricting scope to the login shell retains the intrusion and excludes the unrelated job, and pacing alters presentation only: the alert set is identical.

Table IV reports a live execution against an instrumented kernel, in which four scenario scripts are run beneath the agent and every rule is required to fire.

No throughput or overhead measurement under production load was performed, and none is claimed. What the system does provide is the instrumentation to detect loss when it occurs:

kernel and user-space losses are counted separately and surfaced continuously, and any execution with a non-zero loss count is reported as incomplete coverage rather than presented as a clean result.

XI. Limitations

The exemption of debuggers from the injection rule is based on the command name, which is the first fifteen bytes of the executable name and therefore trivially forged by copying a payload to a suitably named path. The exemption purchases usability rather than security; verifying binary identity through the executable link and a signature or integrity measurement is the correct remedy.

Path matching operates on the unresolved argument string, so a symbolic link, a bind mount, a relative path following a directory change, or access to the underlying block device evades a rule that matches the canonical path. Addressing this properly requires instrumentation at the virtual filesystem or security-module layer, where the resolved inode is available. Relatedly, only openat is instrumented, so a statically linked binary issuing a bare open call is not observed, and the check for set-user-ID execution inspects the path after the execution is observed, leaving a window in which the binary may be replaced.

The most significant coverage gap is the absence of container and namespace awareness. The kernel reports process identifiers from the initial namespace while syscall arguments such as a ptrace target are expressed in the caller's namespace, so a single alert may carry identifiers from two namespaces; and because no control-group identifier is attached to events, the question of which container originated an action cannot be answered. A second gap is the absence of in-kernel filtering: every matching syscall produces a 544-byte event even when the scope filter will discard it immediately, which is the direct and acknowledged cost of the policy-free collection design. Finally, capability-setting calls are not differenced against the bounding set, so a process legitimately dropping privilege is indistinguishable from one acquiring it.

Within the correlator, session boundaries are themselves command-name based, so a process that adopts the name of a session owner divides its own storyline; this costs correlation rather than detection, as every underlying alert still fires. Storylines are keyed by process identifier within the chain window, so on a host that exhausts its identifier space within that window an unrelated process could inherit a storyline.

XII. Future Work

The priorities follow from the limitations rather than from novelty. In-kernel filtering by process identifier and control group, consulted from a map inside each probe, removes the dominant cost of the policy-free design and requires no additional argument with the verifier. Container and namespace awareness, obtained by reading the namespace proxy and control-group identifier through CO-RE relocation, closes the largest coverage gap. Binary identity verification replaces the command-name allowlists and generalises to session-boundary determination. Thereafter: differencing of capability sets, process fork and exit instrumentation for exact tree tracking, which additionally removes the identifier-reuse caveat, and security-module hooks for resolved-path matching.

XIII. Conclusion

KrnlSentry demonstrates that the difficult part of host-based detection is not the collection of syscalls but their interpretation, and that the two are best held strictly apart. Placing observation in the kernel and every judgement in user space produced three results that justify the separation. A sequence attack becomes detectable with confidence, because global event ordering and a syscall-exit probe together permit the detector to verify rather than infer the relationship between the socket and the duplicated descriptor. Alerts become incidents, because reconstructing lineage from the complete stream and escalating on tactic breadth converts four rule hits across four processes into one ordered account while leaving a loud single-tactic workload unescalated. And the detection surface becomes verifiable on an unprivileged workstation, at 94.8% statement coverage, with a golden capture pinning exact outputs.

The system is deliberately explicit about what it does not detect, and its roadmap is ordered by that honesty rather than by novelty. A detection control whose limits are documented is one about which an operator can reason, which is ultimately the property that determines whether a control is useful or merely present.

References

[1] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix for Linux." [Online]. Available: https://attack.mitre.org/matrices/enterprise/linux/

[2] MITRE Corporation, "ATT&CK Tactics: TA0002 Execution, TA0004 Privilege Escalation, TA0005 Defense Evasion, TA0006 Credential Access, TA0011 Command and Control." [Online]. Available: https://attack.mitre.org/tactics/enterprise/

[3] A. Nakryiko, "BPF portability and CO-RE," Feb. 2020. [Online]. Available: https://nakryiko.com/posts/bpf-portability-and-co-re/

[4] A. Nakryiko, "BPF ring buffer," May 2020. [Online]. Available: https://nakryiko.com/posts/bpf-ringbuf/

[5] The Linux Kernel Documentation, "BPF documentation." [Online]. Available: https://docs.kernel.org/bpf/

[6] Cilium authors, "cilium/ebpf: eBPF library for Go." [Online]. Available: https://github.com/cilium/ebpf

[7] IO Visor Project, "BCC: BPF Compiler Collection." [Online]. Available: https://github.com/iovisor/bcc

[8] M. Kerrisk, "ptrace(2), openat(2), setuid(2), capset(2), dup2(2), socket(2), connect(2), execve(2)," Linux Programmer's Manual. [Online]. Available: https://man7.org/linux/man-pages/

[9] S. McCanne and V. Jacobson, "The BSD packet filter: A new architecture for user-level packet capture," in Proc. USENIX Winter Conf., 1993, pp. 259–269.

[10] Red Hat, "Understanding Audit log files (auditd)," Red Hat Enterprise Linux Security Guide. [Online]. Available: https://access.redhat.com/documentation/

[11] Linux Kernel Documentation, "BPF Type Format (BTF)." [Online]. Available: https://docs.kernel.org/bpf/btf.html

[12] B. Gregg, BPF Performance Tools: Linux System and Application Observability. Boston, MA, USA: Addison-Wesley, 2019.